

● ALERT · prod-pipeline-daily · FAILED · 02:07 AM

ON-CALL INTERVIEW EDITION

30 Azure Data Engineer Production Scenarios

With interview-ready answers for the question that decides most rounds:

"What would you do if this happened in production?"

ADF

SHIR

Schema Drift

Databricks · PySpark

Delta Lake

Unity Catalog

Auto Loader

SQL

Swipe to open the incident log →

Definitions get you shortlisted. Scenarios get you hired.

Interviewers hand you a production problem and keep pushing with four follow-ups:

Q1

How would you implement it?

Q2

What happens if it fails?

Q3

How would you optimize it?

Q4

What would you do differently?

HOW TO READ EACH INCIDENT CARD

P1 • CRITICAL

Something broke. Recovery questions.

P2 • HIGH

Data or performance risk.

P3 • DESIGN

Architecture and design questions.

SAY THIS

The one line that makes your answer stand out.

INC-001

ADF

P3 · DESIGN

● RESOLVED

You have 200 source tables. Will you build 200 pipelines?

RESOLUTION

- › No. I'd build a **metadata-driven framework**.
- › A metadata table holds source, target, query, load type and watermark column.
- › `Lookup` reads the metadata, `ForEach` runs one parameterized pipeline per table.
- › Full and incremental loads run through the same framework, driven by load type.

SAY THIS

Adding table #201 should mean one new metadata row, not a new pipeline.

INC-002

SHIR

P3 · DESIGN

● RESOLVED

Your source is an on-premises SQL Server. How do you pull the data?

RESOLUTION

- › Install a **Self-hosted Integration Runtime** on a machine that can reach the SQL Server.
- › Create the SQL Server linked service in ADF and bind it to the SHIR.
- › Flow: ADF → SHIR → SQL Server, then `Copy Activity` lands data in ADLS.
- › In production: firewall rules, Key Vault credentials, and **multi-node SHIR** for high availability.

SAY THIS

A single SHIR node is a single point of failure. Say how you'd avoid it.

INC-003

ADF

P3 · DESIGN

● RESOLVED

Will you load the complete SQL Server table every day?

RESOLUTION

- › No. For large tables I'd use **incremental loading**.
- › Pick a watermark column: `LastModifiedDate`, `UpdatedDate` or an increasing ID.
- › Store the last successful watermark in a control table and fetch only newer rows.
- › Update the watermark only after the load succeeds.

SAY THIS

Full daily loads waste compute and put needless pressure on the source.

INC-004

ADF

P1 · CRITICAL

● RESOLVED

The pipeline fails after processing 80% of the data. What about the watermark?

RESOLUTION

- › The watermark moves **only after full success**.
- › Old watermark = 1000, run reaches 1200 and fails → it stays at 1000.
- › The rerun processes 1001 to 1200 again.
- › So target writes must be **idempotent**: use MERGE or batch-level control to avoid duplicates.

SAY THIS

"Idempotent" is the word the interviewer is waiting to hear.

INC-005

ADF

P1 · CRITICAL

● RESOLVED

You receive 500 files and 50 of them fail. What do you do?

RESOLUTION

- › If files are independent, I **don't fail all 500**.
- › Track status per file: file name, run ID, status, error message, timestamp.
- › Reprocess only the 50 failed files.
- › The 450 successful files are never touched again.

SAY THIS

Recovery should be surgical, not a full rerun.

INC-006

ADF

P3 · DESIGN

● RESOLVED

100 tables must load at 9 AM and another 100 at 11 AM. How do you design it?

RESOLUTION

- › Store scheduling in metadata with a **schedule group** column.
- › The 9 AM trigger selects group 0900, the 11 AM trigger selects group 1100.
- › One parameterized pipeline serves both groups.
- › Scheduling stays centrally controlled with no duplicate pipelines.

SAY THIS

Moving a table to a new slot = updating one metadata value.

INC-007

SCHEMA

P2 • HIGH

● RESOLVED

The source adds a new column. How do you handle schema drift in ADF?

RESOLUTION

- › First ask: is this change **expected or unexpected**?
- › In Mapping Data Flows, enable **schema drift** to carry columns not in the projection.
- › A new nullable column is usually safe to accept.
- › A datatype change or removed required column is **breaking**: validate, alert, monitor.

SAY THIS

Allow drift, but never without validation and monitoring.

INC-008

SCHEMA

P2 • HIGH

● RESOLVED

How would you handle schema drift in Databricks?

RESOLUTION

- › Decide whether the new column should flow downstream at all.
- › **Additive** change: allow controlled Delta schema evolution (e.g. `mergeSchema`).
- › **Breaking** change: validate the schema and raise an alert instead of auto-evolving.
- › For Auto Loader / streaming, track schema location and checkpoints for consistency.

SAY THIS

The key line: "I separate additive changes from breaking changes."

INC-009

SCHEMA

P1 • CRITICAL

● RESOLVED

The source drops a non-nullable column. What now?

RESOLUTION

- › This is a **breaking schema change**.
- › Compare incoming vs expected schema **before** processing.
- › Fail or **quarantine** the batch so incomplete data never reaches Silver or Gold.
- › Log the mismatch and alert the responsible team.

SAY THIS Fail early. Otherwise your BI users discover the problem first.

INC-010

SPARK

P1 • CRITICAL

● RESOLVED

Your 1 TB Databricks job suddenly becomes slow. What do you check?

RESOLUTION

- › I **don't scale the cluster first**.
- › Spark UI: stages, shuffle, task duration, input volume, partition distribution.
- › Look for skew, bad joins, small files, wrong partitioning, unneeded transformations.
- › Fix the root cause: broadcast join, repartition / coalesce, AQE, or Delta layout optimization.

SAY THIS Diagnose before you scale. Bigger clusters hide problems, they don't fix them.

INC-011

SPARK

P2 • HIGH

● RESOLVED

One customer has millions of records, others only a few hundred. What goes wrong?

RESOLUTION

- › Classic **data skew**.
- › Partitioning by customer ID makes one partition far larger than the rest.
- › Most tasks finish fast while one keeps running. Confirm it in Spark UI.
- › Fix with **AQE skew handling**, salting, a better partition key, or broadcast joins.

SAY THIS

Describe the symptom: "199 tasks done, 1 still running."

INC-012

SPARK

P2 • HIGH

● RESOLVED

Your ingestion creates thousands of small files. What do you do?

RESOLUTION

- › Small files mean **metadata overhead** and slow queries.
- › Find the cause first: tiny micro-batches, over-partitioning, high-cardinality partition columns.
- › Fix the write side: optimized writes / auto compaction, repartition before writing.
- › Compact existing data with `OPTIMIZE`, aligned to downstream query patterns.

SAY THIS

Fix why the files are small, then compact. Not the other way around.

INC-013

DELTA

P3 · DESIGN

● RESOLVED

There are duplicates you can't delete, but BI should see only the latest record.

RESOLUTION

- › Keep **full history** in the underlying Delta table.
- › Use `ROW_NUMBER()` partitioned by business key, ordered by update time DESC.
- › Expose only `rn = 1` through a BI view.
- › History is preserved, and BI sees the current business state.

SAY THIS

Separate storage (history) from presentation (latest view).

INC-014

DELTA

P2 · HIGH

● RESOLVED

How would you implement SCD Type 2?

RESOLUTION

- › Add `effective_start_date`, `effective_end_date` and `is_current`.
- › When a customer's address changes, **expire** the active row: set end date, `is_current = false`.
- › **Insert** the new version with `is_current = true`.
- › In Databricks, implement it with **Delta MERGE**.

SAY THIS

Walk through one concrete change (an address update) step by step.

INC-015

DELTA

P2 • HIGH

● RESOLVED

An SCD2 table has 100 columns. How do you detect changes?

RESOLUTION

- › I don't write 100 column comparisons. That's unsustainable.
- › Generate a **hash** of the relevant business columns on source and target.
- › Hashes differ → expire the old version and insert the new one.
- › **Normalize nulls and datatypes** before hashing to avoid false changes.

SAY THIS

Mentioning null normalization shows you've done this in real life.

INC-016

DELTA

P3 • DESIGN

● RESOLVED

Daily customer data contains both new and existing customers.

RESOLUTION

- › Use **Delta MERGE** on the business key.
- › Not matched → insert the record.
- › Matched and changed → update (SCD1) or create a new version (SCD2).
- › This gives an incremental, controlled sync of the target.

SAY THIS

Always state which SCD type the business needs before choosing the logic.

INC-017

DELTA

P2 • HIGH

● RESOLVED

Two pipelines write to the same Delta table at the same time.

RESOLUTION

- › Delta provides **ACID transactions** with **optimistic concurrency control**.
- › At commit, Delta checks for conflicting changes from other writers.
- › On conflict, one write fails instead of silently corrupting the table.
- › By design, avoid needless concurrent writes and add dependencies plus retries.

SAY THIS

Delta protects the table, but good design avoids the conflict in the first place.

INC-018

DELTA

P1 • CRITICAL

● RESOLVED

A user accidentally deletes data from a Delta table.

RESOLUTION

- › Check `DESCRIBE HISTORY` to find the operation and version.
- › Use **time travel** to inspect the previous version.
- › Compare versions and recover in a controlled way (`RESTORE` or targeted re-insert).
- › Then find the root cause and tighten access permissions.

SAY THIS

Recovery is half the answer. Prevention is the other half.

INC-019

DELTA

P2 • HIGH

● RESOLVED

Your source sends INSERT, UPDATE and DELETE events. How do you implement CDC?

RESOLUTION

- › **Bronze**: land raw events with the operation type intact.
- › **Silver**: apply each event by operation type and business key.
- › DELETE: hard delete, or a **soft-delete flag** if history must be kept.
- › If SCD Type 2 is required, combine CDC logic with SCD processing.

SAY THIS

Never lose the operation type in Bronze. You can't rebuild it later.

INC-020

DELTA

P3 • DESIGN

● RESOLVED

Z-ORDER vs Liquid Clustering: how do you decide?

RESOLUTION

- › Start from **query patterns**, table size and how often data changes.
- › **Z-ORDER**: applied via `OPTIMIZE`, suits stable filter columns, needs reruns.
- › **Liquid Clustering**: keys can change without rewrites, suits evolving filters and frequent writes.
- › Measure query performance before and after. Decide on evidence.

SAY THIS

Don't pick a technique because it's new. Pick it because the workload needs it.

INC-021

UNITY

P3 · DESIGN

● RESOLVED

What is an External Location in Unity Catalog?

RESOLUTION

- › Governed access to a **specific cloud storage path**.
- › **Storage Credential** → how we authenticate.
- › **External Location** → which path we allow access to.
- › Access is governed centrally through Unity Catalog grants.

SAY THIS

The two-line "how vs where" explanation is what makes this answer stick.

INC-022

UNITY

P2 · HIGH

● RESOLVED

Different users must see different rows from the same table.

RESOLUTION

- › Use Unity Catalog **row filters** (row-level security).
- › The filter function checks the user's **group membership**.
- › Queries automatically return only the rows that user is authorized to see.
- › No need to maintain separate copies of the data per group.

SAY THIS

One table, many views of it. Duplicating data for security doesn't scale.

INC-023

UNITY

P1 • CRITICAL

● RESOLVED

Your notebook needs database credentials. Where do you store them?

RESOLUTION

- › **Never hardcode** credentials in a notebook.
- › Use **Azure Key Vault** integrated with Databricks via a secret scope.
- › Retrieve secrets at runtime; the password never lives in source code.
- › Apply **least privilege**: only the required identity can read the secret.

SAY THIS

Lead with "never hardcode." It's the first thing they listen for.

INC-024

STREAM

P2 • HIGH

● RESOLVED

An e-commerce app continuously generates order files. Design near-real-time ingestion.

RESOLUTION

- › Files land in ADLS → **Auto Loader** for incremental file discovery.
- › **Bronze** raw → **Silver** cleansed and validated → **Gold** business-ready for BI.
- › Plan for checkpoints, schema evolution, deduplication, data quality and monitoring.
- › For true event streams, use a streaming source instead of forcing Auto Loader.

SAY THIS

Knowing when *not* to use a tool is a senior-level signal.

INC-025

STREAM

P1 • CRITICAL

● RESOLVED

Auto Loader or the streaming job fails halfway. What happens?

RESOLUTION

- › **Checkpointing** stores the progress and state needed for recovery.
- › On restart, the job continues from the last committed point.
- › It doesn't blindly reprocess everything.
- › Retries can still replay data, so keep downstream writes **idempotent**.

SAY THIS Checkpoint + idempotent writes = safe recovery. Say both.

INC-026

SQL

LIVE CODING

● RESOLVED

Find the second-highest salary in every department.

Use `DENSE_RANK()` so tied salaries share the same rank.

```
SELECT *
FROM (
  SELECT *,
    DENSE_RANK() OVER (
      PARTITION BY department
      ORDER BY salary DESC
    ) AS rnk
  FROM employee
) t
WHERE rnk = 2;
```

SAY THIS Know the difference: RANK skips numbers after ties, DENSE_RANK doesn't, ROW_NUMBER never ties.

INC-027

SQL

LIVE CODING

● RESOLVED

Find customers who have never placed an order.

LEFT JOIN customers to orders and keep rows where the order side is NULL.

```
SELECT c.*
FROM customer c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.customer_id IS NULL;
```

SAY THIS Mention NOT EXISTS as an alternative, and avoid NOT IN when the column can be NULL.

INC-028

SQL

LIVE CODING

● RESOLVED

Find duplicate customer IDs.

GROUP BY the ID and use HAVING to keep IDs that appear more than once.

```
SELECT customer_id,
       COUNT(*) AS duplicate_count
FROM customer
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

SAY THIS Expect the follow-up: "Now delete them but keep one." Answer with ROW_NUMBER().

INC-029

SQL

LIVE CODING

● RESOLVED

Find the latest record for every customer.

Use `ROW_NUMBER()` because you need exactly one row per customer.

```
SELECT *
FROM (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY customer_id
      ORDER BY updated_at DESC
    ) AS rn
  FROM customer
) t
WHERE rn = 1;
```

SAY THIS Why not DENSE_RANK? Ties on `updated_at` would return two rows per customer.

INC-030

PROD

P1 · CRITICAL

● RESOLVED

Your pipeline normally takes 30 minutes but suddenly takes 2 hours.

RESOLUTION

- › Compare today's run with the **last successful run**.
- › Check whether **input data volume** increased.
- › Check source query performance, ADF activity durations and Spark UI.
- › Look for shuffle, skew, small files, resource limits, and recent code or schema changes.

SAY THIS Find the bottleneck first. Never just add nodes and hope.

● PAGER · 02:00 AM · prod pipeline FAILED

"Your production pipeline fails at 2 AM. What do you do?"

Answer it like a runbook. Six steps, in this order:

01

Triage

Find the failed activity and capture the exact error.

02

Classify

Source, network, authentication, data quality, transformation or infrastructure?

03

Fix

Resolve the root cause, not just the symptom.

04

Reprocess

Rerun only the failed portion, not the entire pipeline.

05

Validate

Check the target data and watch the next execution.

06

Prevent

Add a safeguard and document the incident.

Answer every scenario in the **same 7 steps.**

1 Requirement**2** Approach**3** Implementation**4** Failure handling**5** Performance**6** Security**7** Monitoring

THEN ANCHOR IT TO ONE END-TO-END FLOW

On-prem SQL Server



SHIR



ADF metadata-driven pipeline



ADLS



Databricks / PySpark



Delta: Bronze → Silver → Gold



Unity Catalog



BI

They're not only checking whether you know Azure services. They're checking whether you can say exactly what you'd do when it breaks.

● STATUS · all incidents resolved

Your next interview is the **real incident.**

Be ready before it starts.

Save

Review the night before

Repost

Help someone on your team
prepare

Comment

The toughest prod issue you've
fixed

Follow **Ashutosh Kadam** for more Azure & Databricks interview prep.